

Frontend Testing mit Jest

Modul: M324 – DevOps-Prozesse mit Tools unterstützen **Autor:** Metehan Celik

Setup

Vor dem Schreiben der Tests wurde die Testumgebung eingerichtet. Folgende Pakete wurden installiert:

```
npm install --save-dev jest react-test-renderer @testing-library/react @testing-library/jest-dom jest-environment-jsdom babel-jest @babel/preset-env @babel/preset-react jest-fetch-mock
```

Die Tests werden im Watch-Mode mit folgendem Befehl gestartet:

```
npm test
```

Dadurch laufen alle Tests automatisch neu, sobald eine Datei gespeichert wird.

Implementierte Tests

Die Tests prüfen die wichtigsten Methoden der App-Komponente: das Rendern, Hinzufügen, Löschen und Bearbeiten von Aufgaben.

Test 1 – Rendern der Überschrift

Prüft ob die ToDo-Liste den Titel korrekt anzeigt.

```
test('renders heading', () => {  
  render(<App />);  
  const headingElement = screen.getByRole('heading', { name: /ToDo Liste/i });  
  expect(headingElement).toBeInTheDocument();  
});
```

Test 2 – Aufgabe hinzufügen

Prüft ob über das Eingabefeld und den "Hinzufügen"-Button eine neue Aufgabe in der Liste erscheint.

```
test('allows user to add a new task', () => {  
  render(<App />);  
  const inputElement = screen.getByLabelText(/Neue Aufgabe hinzufügen/i);  
  const addButtonElement = screen.getByRole('button', { name: /Hinzufügen/i });
```

```
fireEvent.change(inputElement, { target: { value: 'Buy groceries' } });
fireEvent.click(addButtonElement);
expect(screen.getByText('Buy groceries')).toBeInTheDocument();
});
```

Test 3 – Aufgabe löschen

Prüft ob eine Aufgabe nach Klick auf den Löschen-Button verschwindet.

```
test('allows user to delete a task', () => {
  render(<App />);
  // Task hinzufügen
  fireEvent.change(screen.getByLabelText(/Neue Aufgabe hinzufügen/i), { target: {
    value: 'Putzen' } });
  fireEvent.click(screen.getByRole('button', { name: /Hinzufügen/i }));
  // Task löschen
  fireEvent.click(screen.getByRole('button', { name: /Löschen/i }));
  expect(screen.queryByText('Putzen')).not.toBeInTheDocument();
});
```

Test 4 – Aufgabe bearbeiten und speichern

Prüft den kompletten Edit-Flow: Bearbeiten-Button → Text ändern → Speichern.

```
test('allows user to edit and save a task', () => {
  render(<App />);
  fireEvent.change(screen.getByLabelText(/Neue Aufgabe hinzufügen/i), { target: {
    value: 'Kochen' } });
  fireEvent.click(screen.getByRole('button', { name: /Hinzufügen/i }));
  fireEvent.click(screen.getByRole('button', { name: /Bearbeiten/i }));
  fireEvent.change(screen.getByDisplayValue('Kochen'), { target: { value: 'Backen' } });
  fireEvent.click(screen.getByRole('button', { name: /Speichern/i }));
  expect(screen.getByText('Backen')).toBeInTheDocument();
});
```

Bild einfügen (Screenshot des Test-Codes aus App.test.js)

Übersicht aller Tests

#	Test	Geprüfte Methode
1	renders heading	Render
2	allows user to add a new task	handleSubmit
3	input field is cleared after adding a task	handleSubmit

#	Test	Geprüfte Methode
4	does not add empty task	handleSubmit Validierung
5	can add multiple tasks	handleSubmit
6	allows user to delete a task	handleDelete
7	shows edit input when edit button is clicked	handleEdit
8	allows user to edit and save a task	handleSave
9	cancels edit mode without saving changes	Cancel-Button

Testausführung

Alle 9 Tests laufen mit einem einzigen Aufruf von `npm test` und sind grün.

Bild einfügen (Screenshot der Jest-Ausgabe mit allen Tests grün)

Applikation läuft fehlerfrei

Nach den Anpassungen wurde die Applikation gestartet, um sicherzustellen, dass keine Coding-Probleme entstanden sind:

Backend:

```
mvn spring-boot:run
```

Frontend:

```
npm start
```

Bild einfügen (Konsolen-Screenshot des laufenden Frontends, Vite-Server-Output)

Bild einfügen (GUI-Screenshot der laufenden ToDo-App im Browser mit hinzugefügten Tasks)

Anpassungen an App.jsx

Damit die Tests synchron funktionieren, wurde die App-Komponente leicht angepasst:

- **Optimistic Updates:** Tasks werden beim Hinzufügen/Löschen/Bearbeiten sofort im State aktualisiert, bevor die Backend-Anfrage abgeschlossen ist.
- **Label-Anpassung:** Das Eingabefeld wurde mit `htmlFor/id` korrekt mit dem Label verknüpft.
- **Accessibility:** Die Edit- und Delete-Buttons haben `aria-label`-Attribute bekommen für bessere Testbarkeit.

Dies hat die Funktionalität der App nicht verändert, aber die App ist nun besser testbar und reagiert für den User auch schneller.